

Vertiefung zu Images und Container

Einleitung:

Sie haben bereits die Einführung zu Docker gemacht, Container zum Laufen gebracht und selbst Images erstellt.

In diesem Dokument geht es darum, dass Sie ein tieferes Verständnis für Images und Container aufbauen. Dies dient Ihnen später dazu, eigene Images effizienter und ressourcensparender erstellen zu können.

Ziel

- ⇒ Sie können mit dem Basissystem eines Containers umgehen.
- ⇒ Sie können images, container, volumes und den build cache gezielt löschen.
- ⇒ Sie können einen Container verändern und aus dem Container ein neues Image erstellen.
- ⇒ Sie können aus dem Vorgehen vom Verändern des Containers ein Dockerfile ableiten.

Inhalte

Einleitung:	1
Ziel	1
Inhalte	1
Aufgabe Als Vorbereitung aufräumen	2
Ein Container ist mehr als eine Applikation	4
Aufgabe Container aufdatieren, verändern und Image erstellen	5
Aufgabe Dockerfile auf der Basis des vorhergehend erstellen	9
Zusatz-Aufgabe Unterschied CMD und ENTRYPOINT	9
Zusatz-Aufgabe Eigenes cow Image	9

Aufgabe Als Vorbereitung aufräumen

Mittlerweile haben Sie auf Ihrem System schon einiges mit Docker gemacht.
 Und es kann sein, dass sich einiges auf Ihrem System angesammelt hat.

Wieviel Platz wird auf meinem Computer von Docker beansprucht?

Wieviel Platz hat ein Container reserviert?

Wieviel Platz beansprucht ein Image

- **Geben Sie nacheinander ein**

```
docker system df
```

```
docker images
```

```
docker ps -as
```

Interpretation anhand von mongo-express (nicht alle Angaben kopiert):

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
mongo-express	latest	2d2fb2cab8f	17 months ago	136MB

CONTAINER ID	IMAGE	...	NAMES	SIZE
f660c64f664a	mongo-express		mongo-express	168B (virtual 136MB)

- **Finden Sie heraus**, was der Unterschied zwischen den beiden Angaben beim Container ist.

Google: docker reference ps virtual

Writeable / Read Only:

Das Image stellt die Basis für den Container und seine **read-only** daten.

Dazu hat ein Container zusätzlich einen **writeable layer**.

The "size" information shows the amount of data (on disk) that is used for the writable layer of each container. The "virtual size" is the total amount of disk-space used for the read-only image data used by the container and the writable layer.

Aufräumen Container und Images:

Um auf Ihrem System Platz zu schaffen, lohnt es sich nicht mehr benötigte Images und Container zu löschen.

Achten Sie dabei darauf, dass Sie nichts Löschen, was Sie in anderen Modulen noch verwenden.

- **Löschen Sie** nicht mehr benötigte Container.
- **Löschen Sie** nicht mehr benötigte Images

Aufräumen Volumes:

Neben den Containern gibt es auch noch die Volumes.

Doch welches Volume wird von welchem Container benutzt?

Und welches Volume wird von keinem Container mehr benutzt?

- **Erstellen Sie** zu Kontrollzwecken ein Volume, das von keinem Container benutzt wird:

```
docker volume create hello
```

- **Listen Sie** alle Volumes auf:

```
docker volume ls
```

- **Untersuchen Sie** das Volume hello und ein anders Volume, ob es von einem Image verwendet wird?

```
docker volume inspect home
```

```
docker volume inspect <name eines volumes>
```

Hmm so richtig überzeugend ist das nicht.

- **Fragen Sie Google:**

Google: how to determine what containers use the docker volume

Sie finden in der Antwort von *stackoverflow 42857575* zwei *Bashscripts*, die Ihnen helfen die Antwort zu finden.

- **Nutzen Sie** Ihr Können aus Modul 122 und bringen Sie eines der Scripte zum Laufen. Sie müssten damit erkennen, dass das Volume *hello* von keinem Container benutzt wird, andere Volumes werden Ihre Container als 'Nutzer' angeben.

Bevor Sie nun die einzelne Volume Löschen.

- **Schlagen Sie nach**, was bei docker der Befehle *volume prune* macht.
- **Führen Sie** den Befehl aus.
- **Kontrollieren Sie**, ob nun das Volume home gelöscht wurde.
-

Aufräumen Build Cache:

Sie werden sich noch mit dem Build Cache näher auseinandersetzen. Hier mal die Kurzversion, was der Build Cache ist.

Wenn Sie ein Image erstellen, dann speichert Docker die Schritte in seinem Cache.

Erstellen Sie das Image erneut, so wird Docker nur die geänderten Teile des Images neu erstellen, was gleichgeblieben ist, wird aus dem Cache genommen. Der Prozess des Build wird deutlich schneller.

- **Löschen Sie** den Build Cache.

Google: `docker remove build cache`

- **Kontrollieren Sie** mit Hilfe des Befehles ganz am Anfang der Aufgabe Ihr Ergebnis.

Ein Container ist mehr als eine Applikation

Sie haben bereits aus einem Docker Image einen Container erstellt. Dabei stand immer eine im Container gekapselte Anwendung im Vordergrund. Doch da ist noch mehr vorhanden:

Quelle: 26.03.2023 <https://www.ionos.de/digitalguide/server/knowhow/docker-image/>

Das Image enthält sämtliche Informationen und Abhängigkeiten, um einen Container auszuführen. Dazu gehören grundlegende Programm-Bibliotheken und Nutzer-Schnittstellen. Insbesondere ist normalerweise eine Kommandozeilen-Umgebung („Shell“) und eine Implementierung der C-Standard-Bibliothek mit an Bord.

Und je nach Image, sind diese zusätzlichen Programme im Image Programmiersprachen, (PHP, Python, Ruby, Java, JavaScript) Entwicklungstools (node/npm, React, Laravel) , Datenbank-Systeme (MySQL, Postgres, MongoDB, Redis), Webserver (Apache, nginx, lighttpd), Content-Management-Systeme (WordPress, Magento, Ruby on Rails).

Ein Docker Image ist selbst unveränderlich (read-only). Kann aber als Grundlage für den Bau eines weiteren Images verwendet werden.

Ein Container kann sich über die Zeit ändern. Sie können ihn z.B. über die zusätzlichen Programme verändern.

Die Konzepte Docker-Container und Docker-Image sind eng miteinander verknüpft. So kann nicht nur ein Docker-Container aus einem Docker-Image erzeugt werden, sondern auch aus einem laufenden Container ein neues Image. Docker-Image und Docker-Container stehen also in einem ähnlichen Zusammenhang wie Hühnerei und Huhn:

```
docker run <image-id>
```

```
docker commit <container-id>
```

- ⇒ Sie können also, aus einem Image einen Container erstellen. Dann die Teile im Container aufdatieren, ergänzen. Und daraus wieder ein Image erstellen.
- ⇒ Sie sollen diese Möglichkeiten entdecken und anwenden.

Aufgabe Container aufdatieren, verändern und Image erstellen

Sie bauen auf der Basis eines ubuntu images ein neues Image auf.
In diesem neuen Image wird der Wal von Docker Ihren Wa(h)ltext "blubbern".



- ⇒ Sie erleben, wie Sie einen Container ergänzen können.
- ⇒ Sie erleben, wie Sie aus diesem ergänzten Container ein neues Image erstellen.

Basis Image bereitstellen:

- **Hollen Sie sich** vom Dockerhub das Image von ubuntu:18.04 (nicht latest)
- Kontrollieren Sie, ob das Image danach vorhanden ist.
- Starten Sie von Image einen neuen Container.
`docker run ubuntu:18.04`
 Wieso sehen Sie keine Reaktion? Was ist passiert?

Untersuchen Sie den status des containers

`docker ps -as`

Wieso steht der Container auf EXITED?¹

- Löschen Sie den Container.

Bash bei Start des Containers öffnen:

Ein Basis Image biete von sich aus einige Tools an. So auch eine Bash.

- **Starten Sie einen neuen Container und mit die Bash**
`docker run -it --name ubuntu18 ubuntu:18.04 bash`
- Falls Fehlermeldung kommt.. Grund? Ausweg?
The container name "/ubuntu18" is already in use by container ...
- Haben Sie einen Weg gefunden und sehen die Bash?
`root@c5b13fc46485:/#`
- **Geben Sie** `cat /etc/os-release` ein.
 Was passiert?
 Haben Sie Ubuntu 18.04?

¹ Der Container startet und endet auch gleich wieder... Es gibt keine Applikation im Container, die den Container am Laufen hält.

- **Bestimmen Sie die Grösse** von Image und Container? Merken Sie sich die Zahlen.
(Wie man das macht, siehe weiter oben.)

Update und Upgrade:

Als erster Schritt machen Sie nun ein *Update* und *Upgrade* im Container.

Doch was passiert da eigentlich genau?

apt-get update downloads the package lists from the repositories and "updates" them to get information on the newest versions of packages and their dependencies.

apt-get upgrade will fetch new versions of packages existing on the machine if APT knows about these new versions by way of apt-get update.

Quelle: 27.03.2023 <https://askubuntu.com/questions/222348/what-does-sudo-apt-get-update-do>

- **Machen Sie** ein *update* und dann ein *upgrade*
- **Wie hat sich die Grössen verändert?**
Bestimmen Sie die Grössen von Image und Container? Merken Sie sich die Zahlen.
(Wie man das macht, siehe weiter oben.)

Die Application cowsay anpassen.

Zur Erinnerung: Das Ziel ist es, statt der *cow* den *docker wal* zu verwenden.

Und draus dann ein neues Image zu machen

- **Installieren Sie das Package** zu cowsay
`apt-get install -y cowsay --no-install-recommends`

By passing "--no-install-recommends" option, the user lets apt-get know not to consider recommended packages as a dependency to install. This results in smaller downloads and installation of packages .

- **Ausprobieren**
Cowsay liegt nach der Installation im Verzeichnis `/usr/games`
`/usr/games/cowsay hello bbw`

```
root@c5b13fc46485:/# /usr/games/cowsay hallo
< hallo >
-----
      \
       ^__^
      (oo)\_______
         (_____)  )\/\
            ||----w |
            ||     ||
```

Und nun folgen die Schritte zum Umbau auf den *docker wal*.

```
< hello bbw >
-----
      \
       ^__^
      (oo)\_______
         (_____)  )\/\
            ||----w |
            ||     ||
```

- Bild ersetzen
- Symbolischer Link auf das Bild setzen
- Zugriff auf das Programm vereinfachen

- **Zunächst löschen** Sie die alten Hintergrundbilder von cowsay
`rm /usr/share/cowsay/cows/*`
- **Kopieren Sie** die neue Datei `docker.cow` von ausserhalb des Containers in den Container. Stellen Sie dazu die Datei in einem Verzeichnis auf Ihrem Host zur Verfügung.
ACHTUNG den Befehl müssen Sie nicht in der Bash sondern in einer Console auf dem Host machen.
`docker cp docker.cow ubuntu18:/usr/share/cowsay/cows`
- **Kontrollieren Sie** in der Bash, ob es funktioniert, hat
`ls -al /usr/share/cowsay/cows`
- **Umleiten auf docker.cow.**
 Damit `cowsay` standardmässig das neue Bild verwendet, legen Sie einen symbolischen Link an.
 Damit leitet Linux Aufrufe von `default.cow` immer auf `docker.cow` um
`ln -sv /usr/share/cowsay/cows/docker.cow /usr/share/cowsay/cows/default.cow`
-s softlink
-v verbose; print name of files
- Kontrollieren, ob der Link funktioniert.
`ls -l /usr/share/cowsay/cows/default.cow`
- **Aufruf auf über dockersay ermöglichen**
 Damit das Programm `cowsay` über `dockersay` aufgerufen werden kann, setzen Sie einen weiteren Link
`ln -sv /usr/games/cowsay dockersay`
- **Ausprobieren** in der Bash
`/dockersay hello`
- Und auch **von ausserhalb** der Bash ausprobieren
`docker exec -it ubuntu18 /dockersay goodbye`

Den geänderten Container Stoppen und mit der Änderung starten

- Bash schliessen:
`exit`

`docker ps -as`

`docker start`
`docker exec -it ubuntu18 /dockersay goodbye`
- **Wie hat sich die Grösse verändert?**
 Bestimmen Sie die Grösse von Image und Container? Merken Sie sich die Zahlen.
 (Das haben Sie ein paar Seiten weiter oben bereits gemacht.)

Ein neues Image erstellen

- **ID des Containers bestimmen**
`docker ps -as`
Merken Sie sich die ID des Containers
- **Image erstellen:**
`docker commit <id> dockersayingm`
- **Kontrollieren Sie, ob das neue Image erstellt wurde.**

Container starten

- Starten Sie einen Container des neuen Images
`docker run --name dockersays dockersaying /dockersay hello`

```
D:\Dropbox\Projekte\Modul_347\11_dockersay>docker run --name dockersays dockersaying /dockersay hello
```

< hello >

 \--

 ## .
 ## ## ## ==
 ## ## ## ## ===
 { ~~~~~ } =====
 o
 { ~~~~~ }

- ⇒ Sie haben basieren auf einem Basis Image einen Container gestartet.
 - ⇒ Sie haben diesen Container verändert.
 - ⇒ Sie haben aus dem veränderten Container ein neues Image erstellt.
 - ⇒ Sie haben basieren auf dem neuen Image einen Container erstellt.
- **Wozu könnte dieses Vorgehen helfen?**

Aufgabe Dockerfile auf der Basis des vorhergehend erstellen

Das Vorgehen von vorher hat uns geholfen, das neue Verhalten Schritt für Schritt aufzubauen. Jeder Schritt konnte ausprobiert werden.

Nun müssen Sie basieren auf diesem Wissen eine Dockerfile erstellen.

- Notieren Sie sich die einzelnen Schritte, die Sie vorher von Hand ausgeführt haben. Notieren Sie wie Sie den Befehl im Dockerfile schreiben.
- Erstellen Sie das Dockerfile anhand dieser Schritte.

Hilfe? <https://www.ionos.de/digitalguide/server/knowhow/dockerfile/>

Zusatz-Aufgabe Unterschied CMD und ENTRYPOINT

Die RUN-Anweisung führt einen Befehl während des Build-Prozesses aus und legt dabei einen neuen Layer im Docker-Image an. Demgegenüber führt die CMD- bzw. ENTRYPOINT-Anweisung einen Befehl beim Starten des Containers aus. Der Unterschied zwischen den beiden Anweisungen ist subtil:

- *ENTRYPOINT wird genutzt, um einen Container zu erzeugen, welcher beim Starten immer dieselbe Aktion ausführt. Der Container verhält sich also wie eine ausführbare Datei.*
- *CMD wird genutzt, um einen Container zu erzeugen, welcher beim Starten ohne weitere Parameter eine definierte Aktion ausführt. Die voreingestellte Aktion lässt sich durch geeignete Parameter leicht überschreiben.*

Gemeinsam haben beide Anweisungen, dass Sie nur ein einziges Mal pro Dockerfile auftreten dürfen. Es ist jedoch möglich, beide Anweisungen zu kombinieren. In diesem Fall definiert ENTRYPOINT die auszuführende Standard-Aktion beim Starten des Containers, während CMD leicht zu überschreibende Parameter für die Aktion definiert.

Beispiel:

```
ENTRYPOINT ["echo", "Hello"]  
CMD ["World"]
```

Quelle: 03.04.2023: <https://www.ionos.de/digitalguide/server/knowhow/dockerfile/>

Aufgabe:

- Nutzen Sie CMD und ENTRYPOINT so dass

docker run --name dockersays3 <image> goodbye
→ gibt beim Wal die Nachricht goodbye aus

docker run --name dockersays4 <image>
→ gibt beim Wal die Nachricht hello aus.

Zusatz-Aufgabe Eigenes cow Image

- Untersuchen Sie die Datei docker.cow, ändern Sie das Aussehen und probieren Sie Ihr eigenes "Bild" aus.